Cahier des charges

NetMonitor - Application de supervision des équipements réseaux

---

1. Contexte du projet

Les entreprises disposent d'un grand nombre d'équipements interconnectés (routeurs, commutateurs, points d'accès Wi-Fi, serveurs, imprimantes, objets connectés…), parfois répartis sur plusieurs sites. Il n'est pas possible de vérifier manuellement et en permanence chacun d'entre eux.

NetMonitor est une application Java de supervision simplifiée. Elle permettra de recenser les équipements, de suivre leur état (UP, WARNING, DOWN) et leurs indicateurs (CPU, RAM, latence, dernier contact), de détecter automatiquement les anomalies et de permettre aux techniciens de traiter les incidents.

L'application reposera sur une architecture client/serveur : un client graphique JavaFX, un serveur central relié à une base de données, et des agents Java qui simulent les équipements supervisés.

---

2. Acteurs

2.1 Administrateur

- Gère les comptes utilisateurs et leurs rôles
- Ajoute, modifie et supprime les équipements
- Définit les seuils d'alerte (CPU, RAM, latence, délai du heartbeat)
- Dispose de tous les droits du technicien

2.2 Technicien

- Consulte l'état des équipements

- Prend en charge et clôture les incidents
- Crée un incident manuellement si nécessaire

2.3 Observateur

- Consulte les équipements, les événements et les incidents en lecture seule

2.4 Agent (acteur système)

- Représente un équipement supervisé
- Envoie régulièrement un heartbeat et des mesures simulées au serveur

---

3. Besoins fonctionnels

3.1 Fonctionnalités communes

- Authentification (identifiant et mot de passe)
- Consultation du tableau de bord
- Modification de son propre mot de passe
- Déconnexion

3.2 Gestion des équipements

- Ajouter un équipement (nom, adresse IP, type : routeur, switch, point d'accès, serveur, imprimante, autre)
- Afficher la liste des équipements avec leur état et la date du dernier contact
- Rechercher un équipement (par nom, adresse IP, type ou état)
- Modifier les informations d'un équipement
- Supprimer un équipement
- Consulter le détail d'un équipement : dernières mesures, événements, incidents
- Mettre à jour automatiquement l'état d'un équipement :
  - UP : fonctionnement normal
  - WARNING : une mesure dépasse un seuil
  - DOWN : aucun heartbeat reçu depuis un délai défini

3.3 Gestion des utilisateurs

- Créer un compte utilisateur avec un rôle (administrateur, technicien, observateur)
- Modifier le rôle ou réinitialiser le mot de passe d'un utilisateur
- Désactiver ou supprimer un compte
- Contrôler l'accès aux fonctionnalités selon le rôle

3.4 Gestion des incidents

- Générer un événement pour chaque fait important (heartbeat, changement d'état)
- Générer une alerte en cas de situation anormale (ex. CPU > 90 %)
- Créer automatiquement un incident lorsqu'un équipement passe à l'état DOWN
- Afficher la liste des incidents (équipement, gravité, problème, état, date)
- Filtrer les incidents par état, gravité ou équipement
- Prendre en charge un incident (OPEN - IN_PROGRESS)
- Clôturer un incident (IN_PROGRESS - CLOSED)

3.5 Interface graphique

- Première version en ligne de commande (menu texte)
- Version finale en JavaFX :
  - écran de connexion
  - tableau des équipements avec un code couleur (UP vert, WARNING orange, DOWN rouge)
  - formulaires d'ajout et de modification d'un équipement
  - vue des incidents avec les actions « Prendre en charge » et « Clôturer »
  - vue du journal des événements
  - vue de gestion des utilisateurs (administrateur uniquement)
- Rafraîchissement automatique des données

---

4. Besoins non fonctionnels

4.1 Persistance

- Stockage en mémoire lors de la première étape

- Sauvegarde par sérialisation Java dans un fichier
- Version finale : base de données relationnelle accessible via JDBC
- Seul le serveur accède à la base de données

4.2 Robustesse

- Validation des saisies (nom non vide, adresse IP valide, doublons interdits)
- Gestion des erreurs par exceptions et messages compréhensibles pour l'utilisateur
- La déconnexion d'un client ou d'un agent n'arrête pas le serveur
- Un message invalide reçu par le serveur est rejeté sans bloquer les autres connexions
- Gestion de plusieurs clients et agents simultanément sans corruption des données
- Reconnexion automatique du client et de l'agent en cas de perte du serveur

4.3 Sécurité

- Mots de passe hachés, jamais stockés en clair
- Requêtes préparées (PreparedStatement) pour éviter les injections SQL
- Vérification des droits côté serveur

4.4 Performance

- Détection rapide d'un équipement indisponible
- Interface fluide : aucun traitement long sur le thread JavaFX

4.5 Contraintes techniques

- Langage Java, programmation orientée objet
- Interface graphique JavaFX
- Base de données via JDBC
- Communication réseau par sockets Java
- Threads pour les traitements concurrents
- Code organisé en couches (modèle, accès aux données, réseau, interface)

---

5. Règles de gestion

- Un utilisateur doit être authentifié pour accéder à l'application
- Chaque utilisateur possède un seul rôle
- Le nom et l'adresse IP d'un équipement sont uniques
- Un nouvel équipement est à l'état DOWN tant qu'aucun heartbeat n'a été reçu
- Un équipement est déclaré DOWN si aucun heartbeat n'est reçu pendant 30 secondes
- Un équipement passe à l'état WARNING si CPU > 90 %, RAM > 90 % ou latence > 200 ms
- Un équipement revient à l'état UP lorsque les mesures redeviennent normales
- Chaque changement d'état génère un événement horodaté
- Un seul incident ouvert par équipement et par problème (pas de doublon)
- Un incident suit le cycle OPEN - IN_PROGRESS - CLOSED ; un incident clôturé ne peut pas être rouvert
- Seuls les techniciens et administrateurs peuvent traiter les incidents
- Seul l'administrateur peut gérer les utilisateurs et supprimer des équipements

---

6. Questions de réflexion

6.1 Actions nécessitant une authentification

- Toutes les actions de l'application

6.2 Actions réservées à certains rôles

- Gestion des utilisateurs : administrateur
- Ajout et suppression d'équipements : administrateur
- Traitement des incidents : technicien et administrateur

6.3 Données à conserver dans l'historique

- Mesures reçues (CPU, RAM, latence) avec leur date
- Changements d'état des équipements

- Alertes générées
- Incidents (création, prise en charge, clôture, technicien concerné)
- Connexions des utilisateurs

---

7. Conclusion

Ce projet permet de mettre en pratique des compétences essentielles en développement Java : modélisation orientée objet, gestion des utilisateurs et des droits, persistance des données, communication réseau, programmation concurrente et interface graphique. Il constitue une base pour un véritable outil de supervision d'infrastructure.